MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on laboratory work no. 0

of Techniques and Mechanisms of Software Design

Topic: SOLID Principles in Swift (SRP, OCP, DIP)

Done by: st. gr. FAF-243, Augustin Carazanu
Checked by: *assist. univ. Daniel Istrati*
FCIM, UTM

Chișinău – 2026

1. Objectives

- Study the SOLID principles of object-oriented design.
- Choose three principles and implement each in Swift, first in a form that violates the principle and then in a refactored form that respects it.
- Compare the two versions and show, through code and unit tests, what the refactoring gains in maintainability, extensibility and testability.

2. Theoretical background

SOLID is an acronym introduced by Robert C. Martin for five design principles that keep object-oriented code easy to change: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation and Dependency Inversion. Their common goal is low coupling and high cohesion: a change in one requirement should touch as few classes as possible, and a class should be understandable on its own.

This laboratory work covers three of them:

- **Single Responsibility Principle (SRP)** — a class should have only one reason to change, i.e. it should be responsible to a single actor or concern.
- **Open/Closed Principle (OCP)** — software entities should be open for extension but closed for modification. New behaviour is added by writing new code, not by editing code that already works.
- **Dependency Inversion Principle (DIP)** — high-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details.

Swift supports these principles directly through protocols (abstractions), protocol-oriented programming, value types, extensions and initializer-based dependency injection.

3. Implementation

All examples share one domain: a small online bookstore that receives orders, takes payments and notifies customers. The project is a Swift Package with three targets: SOLIDCore (library with all examples), SOLIDDemo (executable that prints the before/after behaviour) and SOLIDTests (XCTest unit tests). Each principle has a separate folder with a Before and an After file; the two versions are kept in separate namespaces (SRPBefore / SRPAfter, etc.) so they can be compiled and run side by side.

```
import Foundation

/// Shared domain model used by all examples: a small online bookstore.
public struct OrderItem: Equatable {
    public let name: String
    public let price: Double
    public let quantity: Int
```

```
    public init(name: String, price: Double, quantity: Int) {
        self.name = name
        self.price = price
        self.quantity = quantity
    }
}

public struct Order: Equatable {
    public let id: Int
    public let customerEmail: String
    public let items: [OrderItem]

    public init(id: Int, customerEmail: String, items: [OrderItem]) {
        self.id = id
        self.customerEmail = customerEmail
        self.items = items
    }
}

/// Formats an amount as "123.45 MDL".
public func money(_ amount: Double) -> String {
    String(format: "%.2f MDL", amount)
}
```

Listing 1 — Shared domain model (Models.swift)

3.1 Single Responsibility Principle

Before — the problem

OrderManager does four unrelated jobs inside one method: it validates the order, calculates the price with a discount, stores the order and builds the confirmation email. A change in the discount policy, the database, the validation rules or the email text all require editing the same class, and none of these parts can be tested or reused separately.

```
// Single Responsibility Principle — BEFORE
//
// `OrderManager` validates the order, calculates the price, stores the order
// and emails the customer. It has four reasons to change: a new validation
// rule, a new discount policy, a new database, or a new email template.

public enum SRPBefore {

    public final class OrderManager {
        public private(set) var savedOrders: [Order] = []
        public private(set) var sentEmails: [String] = []

        public init() {}

        public func process(_ order: Order) -> String {
            // Responsibility 1: validation
            if order.items.isEmpty {
                return "Error: order #\(order.id) has no items"
            }
            if !order.customerEmail.contains("@") {
                return "Error: invalid email '\(order.customerEmail)'"
            }

            // Responsibility 2: pricing (10% discount over 500 MDL)
```

```
            var total = 0.0
            for item in order.items {
                total += item.price * Double(item.quantity)
            }
            if total > 500 {
                total *= 0.9
            }

            // Responsibility 3: persistence
            savedOrders.append(order)

            // Responsibility 4: notification
            let email = "To: \(order.customerEmail) | Order #\(order.id) confirmed, total \
(money(total))"
            sentEmails.append(email)

            return "Order #\(order.id) processed, total \(money(total))"
        }
    }
}
```

Listing 2 — SRP violated (SRPBefore.swift)

After — the solution

The responsibilities are split into four focused types. OrderValidator only validates and reports problems through a typed OrderError. PriceCalculator only computes the total, and its discount threshold and rate are now configurable. OrderRepository only stores orders and OrderNotifier only builds and sends confirmations. OrderService coordinates the use case without containing any business rule itself.

```
// Single Responsibility Principle — AFTER
//
// Each class now has exactly one reason to change. `OrderService` only
// coordinates the workflow; the actual work is delegated.

public enum SRPAfter {

    public enum OrderError: Error, Equatable, CustomStringConvertible {
        case noItems(orderID: Int)
        case invalidEmail(String)

        public var description: String {
            switch self {
            case .noItems(let id): return "Error: order #\(id) has no items"
            case .invalidEmail(let email): return "Error: invalid email '\(email)'"
            }
        }
    }

    /// Changes only when validation rules change.
    public struct OrderValidator {
        public init() {}

        public func validate(_ order: Order) throws {
            guard !order.items.isEmpty else { throw OrderError.noItems(orderID: order.id) }
            guard order.customerEmail.contains("@") else { throw
OrderError.invalidEmail(order.customerEmail) }
```

```
        }
    }

    /// Changes only when the pricing / discount policy changes.
    public struct PriceCalculator {
        public let discountThreshold: Double
        public let discountRate: Double

        public init(discountThreshold: Double = 500, discountRate: Double = 0.10) {
            self.discountThreshold = discountThreshold
            self.discountRate = discountRate
        }

        public func total(for order: Order) -> Double {
            let subtotal = order.items.reduce(0) { $0 + $1.price * Double($1.quantity) }
            return subtotal > discountThreshold ? subtotal * (1 - discountRate) : subtotal
        }
    }

    /// Changes only when the storage mechanism changes.
    public final class OrderRepository {
        public private(set) var orders: [Order] = []
        public init() {}

        public func save(_ order: Order) {
            orders.append(order)
        }
    }

    /// Changes only when the notification format/channel changes.
    public final class OrderNotifier {
        public private(set) var sentEmails: [String] = []
        public init() {}

        public func sendConfirmation(for order: Order, total: Double) {
            sentEmails.append("To: \(order.customerEmail) | Order #\(order.id) confirmed,
total \(money(total))")
        }
    }

    /// Coordinates the use case; contains no business logic of its own.
    public final class OrderService {
        private let validator: OrderValidator
        private let calculator: PriceCalculator
        public let repository: OrderRepository
        public let notifier: OrderNotifier

        public init(validator: OrderValidator = OrderValidator(),
                    calculator: PriceCalculator = PriceCalculator(),
                    repository: OrderRepository = OrderRepository(),
                    notifier: OrderNotifier = OrderNotifier()) {
            self.validator = validator
            self.calculator = calculator
            self.repository = repository
            self.notifier = notifier
        }

        public func process(_ order: Order) -> String {
            do {
                try validator.validate(order)
```

```
            } catch let error as OrderError {
                return error.description
            } catch {
                return "Error: \(error)"
            }
            let total = calculator.total(for: order)
            repository.save(order)
            notifier.sendConfirmation(for: order, total: total)
            return "Order #\(order.id) processed, total \(money(total))"
        }
    }
}
```

Listing 3 — SRP respected (SRPAfter.swift)

3.2 Open/Closed Principle

Before — the problem

PaymentProcessor decides the fee and the confirmation text with a switch over the PaymentType enum. Adding a new method (for example MPay) means editing the enum and every switch statement in the class. Code that is already tested gets modified, and with more payment methods the switches keep growing.

```
// Open/Closed Principle — BEFORE
//
// Every new payment method forces us to edit the `PaymentType` enum AND
// every `switch` inside `PaymentProcessor`. The class is not closed for
// modification, and each edit risks breaking methods that already work.

public enum OCPBefore {

    public enum PaymentType {
        case card
        case payPal
        case cash
        // Adding MPay means: add a case here, then edit both switches below.
    }

    public final class PaymentProcessor {
        public init() {}

        public func fee(for amount: Double, type: PaymentType) -> Double {
            switch type {
            case .card:   return amount * 0.015
            case .payPal: return amount * 0.034 + 5
            case .cash:   return 0
            }
        }

        public func pay(_ amount: Double, type: PaymentType) -> String {
            let total = amount + fee(for: amount, type: type)
            switch type {
            case .card:   return "Charged \(money(total)) to the bank card"
            case .payPal: return "Sent \(money(total)) through PayPal"
            case .cash:   return "Collect \(money(total)) in cash on delivery"
            }
        }
    }
```

```
}
```

Listing 4 — OCP violated (OCPBefore.swift)

After — the solution

The variation point is extracted into the PaymentMethod protocol. Each payment method is a separate type that knows its own fee and confirmation message. PaymentProcessor works only with the protocol and no longer contains a switch, so it is closed for modification.

```
// Open/Closed Principle — AFTER
//
// `PaymentProcessor` depends on the `PaymentMethod` abstraction. New payment
// methods are added as new types (see MPayPayment.swift) — the processor and
// the existing methods are never edited again.

/// The extension point. Any new payment method just conforms to it.
public protocol PaymentMethod {
    var name: String { get }
    func fee(for amount: Double) -> Double
    func confirmation(for total: Double) -> String
}

public enum OCPAfter {

    public struct CardPayment: PaymentMethod {
        public let name = "Card"
        public init() {}
        public func fee(for amount: Double) -> Double { amount * 0.015 }
        public func confirmation(for total: Double) -> String { "Charged \(money(total)) to
the bank card" }
    }

    public struct PayPalPayment: PaymentMethod {
        public let name = "PayPal"
        public init() {}
        public func fee(for amount: Double) -> Double { amount * 0.034 + 5 }
        public func confirmation(for total: Double) -> String { "Sent \(money(total)) through
PayPal" }
    }

    public struct CashPayment: PaymentMethod {
        public let name = "Cash"
        public init() {}
        public func fee(for amount: Double) -> Double { 0 }
        public func confirmation(for total: Double) -> String { "Collect \(money(total)) in
cash on delivery" }
    }

    /// Closed for modification: no switch, no knowledge of concrete methods.
    public final class PaymentProcessor {
        public init() {}

        public func pay(_ amount: Double, with method: PaymentMethod) -> String {
            let total = amount + method.fee(for: amount)
            return "[\(method.name)] " + method.confirmation(for: total)
        }
    }
}
```

Listing 5 — OCP respected (OCPAfter.swift)

To show extension without modification, MPay support is added afterwards in a new file. No existing line was changed. The unit tests go further and define a GiftCardPayment type inside the test file, which the processor accepts as well.

```
// Open/Closed Principle — EXTENSION
//
// A new requirement: support MPay (the Moldovan government payment service).
// It is added in its own file. No existing line of code was modified.

public extension OCPAfter {

    struct MPayPayment: PaymentMethod {
        public let name = "MPay"
        public init() {}
        public func fee(for amount: Double) -> Double { 2 }
        public func confirmation(for total: Double) -> String { "Paid \(money(total)) via
MPay" }
    }
}
```

Listing 6 — Extending the system with a new method (MPayPayment.swift)

3.3 Dependency Inversion Principle

Before — the problem

The high-level NotificationService creates a concrete EmailSender inside itself. The business policy (“notify the customer when the order ships”) is tied to one delivery technology. Sending SMS or using several channels requires rewriting the service. A unit test of the service would also always go through the real sender.

```
// Dependency Inversion Principle — BEFORE
//
// The high-level `NotificationService` creates and depends directly on the
// low-level `EmailSender`. Switching to SMS, sending through several
// channels, or replacing the sender with a fake in unit tests is impossible
// without rewriting the service.

public enum DIPBefore {

    /// Low-level detail.
    public final class EmailSender {
        public private(set) var outbox: [String] = []
        public init() {}

        public func sendEmail(to address: String, body: String) {
            outbox.append("EMAIL -> \(address): \(body)")
        }
    }

    /// High-level policy, hard-wired to the detail above.
    public final class NotificationService {
        public let sender = EmailSender()          // concrete, created inside
        public init() {}
```

```
        public func notifyShipped(orderID: Int, contact: String) {
            sender.sendEmail(to: contact, body: "Order #\(orderID) has been shipped")
        }
    }
}
```

Listing 7 — DIP violated (DIPBefore.swift)

After — the solution

The MessageSender protocol is an abstraction owned by the high-level layer. NotificationService receives an implementation through its initializer (constructor injection) and knows nothing about email or SMS. The low-level classes EmailSender and SMSSender depend on the same abstraction. A MultiChannelSender composite shows that combined behaviour needs no change in the service, and a mock sender can be injected in tests.

```
// Dependency Inversion Principle — AFTER
//
// Both the high-level `NotificationService` and the low-level senders depend
// on the `MessageSender` abstraction. The concrete sender is injected
// through the initializer, so it can be swapped (email, SMS, several
// channels, a test double) without touching the service.

/// The abstraction owned by the high-level module.
public protocol MessageSender: AnyObject {
    func send(to recipient: String, body: String)
}

public enum DIPAfter {

    public final class EmailSender: MessageSender {
        public private(set) var outbox: [String] = []
        public init() {}
        public func send(to recipient: String, body: String) {
            outbox.append("EMAIL -> \(recipient): \(body)")
        }
    }

    public final class SMSSender: MessageSender {
        public private(set) var outbox: [String] = []
        public init() {}
        public func send(to recipient: String, body: String) {
            outbox.append("SMS -> \(recipient): \(body)")
        }
    }

    /// Composite: sends through several channels, still just a MessageSender.
    public final class MultiChannelSender: MessageSender {
        private let senders: [MessageSender]
        public init(_ senders: [MessageSender]) { self.senders = senders }
        public func send(to recipient: String, body: String) {
            senders.forEach { $0.send(to: recipient, body: body) }
        }
    }

    /// High-level policy: knows only the abstraction.
    public final class NotificationService {
        private let sender: MessageSender
```

```
        public init(sender: MessageSender) {     // dependency injection
            self.sender = sender
        }

        public func notifyShipped(orderID: Int, contact: String) {
            sender.send(to: contact, body: "Order #\(orderID) has been shipped")
        }
    }
}
```

Listing 8 — DIP respected (DIPAfter.swift)

4. Comparison

Principle	Before	After	Benefit
SRP	One class: validation, pricing, storage, email	Four single-purpose types + coordinating service	Localized changes, each part testable and reusable
OCP	switch over PaymentType in every method	PaymentMethod protocol, one type per method	New methods (MPay, GiftCard) added with zero edits
DIP	Service instantiates concrete EmailSender	Service depends on MessageSender, injected via init	Swap email/SMS/multi-channel, mock in tests

Table 1 — Before/after comparison

5. Testing and results

The SOLIDTests target uses XCTest to check that the refactored designs behave correctly and can be extended:

- SRP — OrderValidator rejects an empty order with the right error, PriceCalculator applies the 10% discount above 500 MDL (2 × 300 → 540 MDL), and OrderService stores the order and sends exactly one confirmation.
- OCP — a GiftCardPayment type declared only in the test file works with PaymentProcessor without changes, and MPay charges a flat 2 MDL fee.
- DIP — a MockSender injected into NotificationService records the recipient and the message. This test could not be written for the Before version.

Running swift run SOLIDDemo prints the before and after behaviour for each principle. Both versions produce the same business results, which confirms that the refactoring kept the behaviour unchanged while changing the structure:

```
========== SRP - BEFORE (one class, four responsibilities) ==========
Order #1 processed, total 693.00 MDL
Error: order #2 has no items
Error: invalid email 'augustin.example.md'

========== SRP - AFTER (validator, calculator, repository, notifier) ==========
Order #1 processed, total 693.00 MDL
Error: order #2 has no items
Error: invalid email 'augustin.example.md'
Saved orders: [1]
To: augustin@example.md | Order #1 confirmed, total 693.00 MDL
```

```
========== OCP - BEFORE (switch over PaymentType) ==========
Charged 1015.00 MDL to the bank card
Sent 1039.00 MDL through PayPal
Collect 1000.00 MDL in cash on delivery

========== OCP - AFTER (PaymentMethod protocol, MPay added without edits) ==========
[Card] Charged 1015.00 MDL to the bank card
[PayPal] Sent 1039.00 MDL through PayPal
[Cash] Collect 1000.00 MDL in cash on delivery
[MPay] Paid 1002.00 MDL via MPay

========== DIP - BEFORE (service creates EmailSender itself) ==========
EMAIL -> augustin@example.md: Order #1 has been shipped

========== DIP - AFTER (sender injected through MessageSender) ==========
EMAIL -> augustin@example.md: Order #1 has been shipped
EMAIL -> customer: Order #3 has been shipped
SMS -> +373 69 000 000: Order #1 has been shipped
SMS -> customer: Order #3 has been shipped
```

Listing 9 — Program output (swift run SOLIDDemo)

Conclusion

In this laboratory work three SOLID principles were applied in Swift on a single bookstore domain, each shown as a violating version and a refactored one.

Applying SRP turned one large class with four reasons to change into small, cohesive types, so a new discount rule or storage engine now affects only one file. OCP replaced type-based switch statements with polymorphism through a protocol. Adding MPay and a gift-card method needed new code only, and the processor that was already tested stayed unchanged. DIP reversed the dependency direction, so the notification policy depends on an abstraction and the concrete senders are injected. This made multi-channel delivery possible without changing the service, and made the service unit-testable with a mock.

Swift protocols, extensions and initializer injection make these principles natural to express. The main trade-off is a larger number of small types. In return, every requirement change stays local, and the cost of a change no longer grows with the size of the system.

Bibliography

- Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
- Martin, R. C. Agile Software Development: Principles, Patterns, and Practices. Prentice Hall, 2002.
- Apple Inc. The Swift Programming Language — Protocols. docs.swift.org.

Repository: https://github.com/Afyze/tmps-2026-AugustinCarazanu